

第 6 阶段:Agentic REPL 自愈循环

Stage 6 — Self-Healing Code-Generation Loop · 设计文档

1. 任务、内容、以及为什么这么做

1.1 这个阶段在整条 pipeline 里扮演什么角色

Stage 5 提供了一个安全的代码执行 sandbox(gVisor + AST 双层隔离)。Stage 6 的任务是 把这个 sandbox 用起来——让 LLM

自动生成代码、扔进去跑、看结果、出错时自己修、直到成功或彻底失败为止。这是把一个会执行代码 的系统变成 会解决问题 的系统的 关键一跃。

1.2 具体任务清单

#	任务	所属模块
T1	接收用户自然语言问题	main.py
T2	调用 LLM 生成只读 SQL	generator.generate_sql()
T3	主进程执行 SQL 拿到 data	generator.run_sql()
T4	把 data 喂给 LLM,生成 Python 分析代码	generator.generate_code()
T5	注入 data 为 JSON 字面量,提交 Sandbox 执行	loop._inject_data()
T6	失败 → 回喂 stderr 给 LLM 重试	loop.run()
T7	最多 3 次重试,仍失败带诊断信息返回	loop.run()

1.3 为什么必须做自愈循环(不是可选)

LLM 生成的代码 首次成功率约 60-80%——常见失败模式:列名错(LLM 记成了类似但不存在的字段)、类型错(把字符串当整数比)、空集处理缺失(忘记判 len(data)==0)、imports 漏写、SQL 结果结构跟代码预期不符。

不做自愈循环,这些错误 会直接成为用户面前的 traceback——产品立刻退化为 demo 玩具。做了自愈,绝大多数失败会在 1-2 次重试内消化,用户根本看不见。

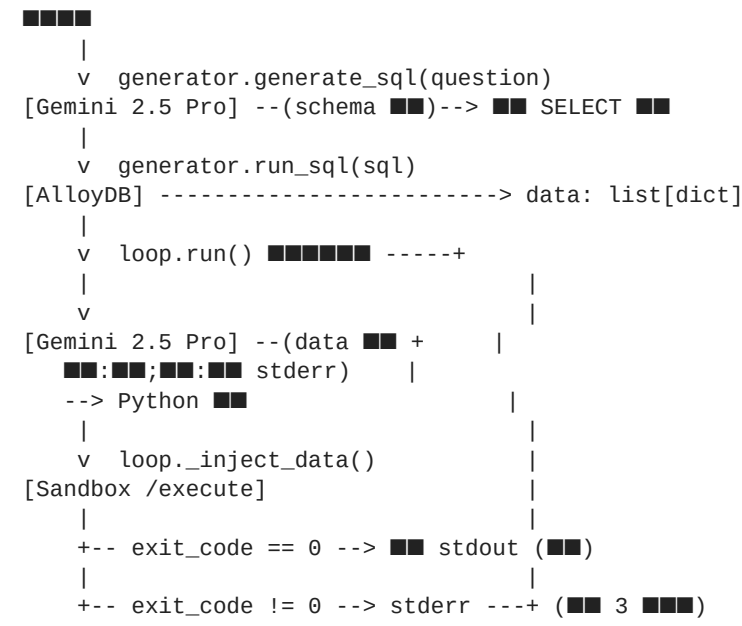
1.4 和 Pandora 的对位

竞品 Pandora 的 pipeline trace 里有一步叫 Reasoning in sandbox(单步 47.7 秒),那本质上就是一个 ReAct 风格的 LLM-in-loop 自愈循环。本阶段做的是同一类东西,但有两个差异点:

- Pandora 是 命令式 agent loop(LLM 在 sandbox 里边跑边想);我们是 声明式——LLM 一次性生成完整 Python 脚本,失败再整篇重写。
- Pandora 的 trace 是封装的黑盒(用户只看到一个名字);我们的每次尝试都留底(代码 + stderr + retry 原因),可审计。

2. 现有实现细节

2.1 整体数据流



2.2 文件分工

文件	行	职责
repl/main.py	61	交互式 CLI 入口,循环读问题、调 run()、打印结果
repl/loop.py	106	run() 主控:SQL → data → 注入 → sandbox → 重试
repl/generator.py	167	CodeGenerator:封装 Gemini 调用与 prompt 模板
repl/__init__.py	0	包标记(空)
sandbox/client.py	104	复用 Stage 5 提供的 HTTP client

2.3 关键设计决策

① 为什么 SQL 和 Python 分两步生成?

一步法(让 LLM 直接生成可执行 Python,里面嵌 SQL)看起来简洁,但 耦合度过高——SQL 错和 Python 错的 traceback 不易区分,LLM 修起来容易越改越错。分两步把 数据获取 和 数据分析 解耦,各自独立失败模式、独立修复策略。

② 为什么把 data 序列化成 JSON 字面量注入,而不是文件传递?

Sandbox 没有网络,也不能挂载任意路径。最干净的传递方式就是把 data 作为代码的一部分。代价是 data 不能太大(单次约束在数 MB 级别),适合 Stage 4 已经做过初步聚合 / 过滤之后的中间结果。

③ 为什么 LLM 历史只在 code_history 里维护?

SQL 阶段是无状态单次调用,失败就直接整轮失败。Python 阶段才是真正的多轮重试,因此只在 generator 里单独维护一份 code_history,SQL 不参与历史。

3. 可改进方向(按 ROI 排序)

优先级	改进项	一句话说明	估时
P0 ★	SQL 阶段自愈	SQL 错就直接死;应该把 SQL 报错也回喂 LLM,重试 1-2 次	0.5d
P0 ★	结构化 Trace 输出	仿 Pandora 9 步进度条,把 routing/sql/exec/retry 用 JSON 事件流推出	1d
P0 ★	history 边界	code_history 无限增长,3 次重试后 prompt 上万 tokens;只保留最近 2 轮	0.2d
P1	错误分类	把 stderr 分成 syntax / runtime / semantic 三类,采用不同修复策略	1d
P1	Schema as tool	让 LLM 可主动调 get_schema(table) 拿字段,而不是一开始就全部塞 prompt	1d
P1	Artifact 持久化	每次 run 写一个 run_<hash>.json,记录 question/sql/data/code/result	0.5d
P2	Verifier pass	成功后再让 LLM 检查"答案像不像合理结果",防止 silently wrong	1d
P2	长生 kernel	替代每次 fresh subprocess,获得 Pandora 那种跨轮 df 复用	1-2w
P2	Streaming UI	FastAPI + SSE 把 trace 流式吐给浏览器(对接 Stage 10)	1w

3.1 本次提交先做的三件(P0)

结合「先跑通 demo + 不过度工程」原则,本次提交 只做 P0 三项:

- 新增 repl/trace.py——结构化 trace 事件,demo 输出立刻有 Pandora 那种 9 步进度感
- loop.py 加 SQL 自愈,SQL 错也能在 1-2 次重试内恢复
- generator.py 加 history 裁剪,prompt 不会因为重试无界膨胀

3.2 长期改进的判断标准(什么时候上 P1 / P2)

取决于 demo 之后两类信号:

- 用户提问失败率——如果 >15% 的问题在 3 次重试后仍失败,优先做 P1 的错误分类 + schema-as-tool
- 跨轮上下文需求——如果用户开始问『接着刚才那个再筛一下』这类问题,优先做 P2 的长生 kernel
- 客户实际使用强度——日均 50+ 次提问才值得投入 Streaming UI 和 artifact 持久化